

实验运行截图：

```
hwz123456@hwz:~/ICS-Lab1$ ./check_ops.py bits.c
mul5Sat: 27/30 operators
classifyAdd3: 26/52 operators
floatScaleThreeHalves: 42/60 operators
floatRoundEven: 25/65 operators
float_i2f: 30/40 operators
bitCount: 28/40 operators
bitReverse: 32/34 operators
All 19 functions passed operator checks.
```

```
hwz123456@hwz:~/ICS-Lab1$ ./btest
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

P1.直接写出即可

P2.考虑到只给了 $\sim$ &, 而只有&包含两个数的运算。首先, 先列出 $\wedge$ 的运算表, 为 $1\wedge 1=0, 1\wedge 0=1, 0\wedge 1=1, 0\wedge 0=0$, 所以只要像构建加法表一样构建出这样的运算表即可。由于&得到的是三个0, 一个1, 而 $\sim$ 又只能对全体执行, 所以只靠一个x, 一个y和 $\sim$ &去组合不能得到 $\wedge$ 的结果, 只能考虑一对x, y的组合, 经过尝试之后就可以得到构建 $\sim((x \& \sim y) \& \sim(x \& y))$ 就可以得出结果。

P3.首先常规负数转换成整数容易写出, 但是如果不加if就不知道是输入的数是负数还是整数。所以要区分负数和正数, 首先负数首位是1, 其次负数在 $\gg$ 后会得到1的补位, 所以只需要将负数 $\gg 31$ 就可以构建11111111111111111111111111111111掩码, 然后正数会得到0, 所以这样, 首先构建 $a=x \gg 31$, 然后再进行 $(\sim x + 1) \& \text{mask}$ 的操作即可。

P4.需要复制scr处数字到dst, 只需要把scr处的数字提取出来, 然后清空dst处的数字, 然后全部加起来或进行|运算就可以了。中间用掩码实现。所以只要 $\text{int } a = \text{src} \ll 3; \text{int } b = \text{dst} \ll 3; \text{int } \text{byte} = (x \gg a) \& 0xFF; \text{int } c = 0xFF \ll b; \text{return } (x \& \sim c) | (\text{byte} \ll b);$ 先构建scr处的数字, 然后再清空x中dst处的数字, 再进行|即可。

P5. $\gg$ 是算术右移, 负数高位会补1。所以需要构建掩码来使高位变成0, 对正数也成立。那么通过 $\text{int } a = \sim(((1 \ll 31) \gg n) \ll 1);$ 构建高位掩码即可。

P6.在整个数左移或右移4时可以实现字节内部高位低位互换的一部分, 而构建掩码清除移动另一半就可以实现互换的一半, 即如字节内部高位移到低位, 原来的低位清除。故掩码 $\text{int } a = 0x0F | (0x0F \ll 8) | (0x0F \ll 16) | (0x0F \ll 24);$ 进行字节内部高位向低位和低位向高位的移动后整合在一起即可。 $((x \& a) \ll 4) | ((x \gg 4) \& a)$

P7.观察发现在加上1后最低位0就会变成1, 然后再加上1后最低位0会变成1。为了不产生新的0, $\text{int } y = x | (x + 1);$ 然后进行 $\sim y \& (y + 1);$ 找寻到y最低0的具体位置, 即为x第二最低0的具体位置。

P8. $\wedge$ 运算会消除偶数次的1, 运用 $\wedge$ 运算消除偶数次1后得到1个1或没有1就可以判断1出现次数的奇偶。运用x自身的数字进行 $\wedge$ 运算即可消除偶数次1, 所以每次二分x进行运算得到最终1的个数。 $x = x \wedge (x \gg 16);$

$x = x \wedge (x \gg 8);$

```

x = x ^ (x >> 4);
x = x ^ (x >> 2);
x = x ^ (x >> 1);
return ~x & 1;

```

P9.每 32 次移动产生循环，先取 $n \bmod 32$ ，再进行同 P5 的右移，最后进行类似 P5 的左移，都采用掩码消除多余位，最后合并即可。int rotateRightBits(int x, int n) {

```

    int s = n & 31;
    int a = ~((1 << 31) >> s) << 1;
    int right = (x >> s) & a;
    int r = (32 + ~s + 1) & 31;
    return right | (x << r);
}

```

P10.首先看 x 除以 2^n 的商和余数，根据规则，余数和商判断应该舍入哪个数字。如果余数大于一半，进位，如果余数等于一半，看商是奇数还是偶数判断。所以构造 $\text{int add} = \text{half} - 1 + (q \& 1)$;这样刚好可以满足规则，加入 return 语句中就可以实现结果。

```

int half = 1 << (n + ~0);
int q = x >> n;
int add = half + ~0 + (q & 1);
return ((x + add) >> n) << n;

```

P11.由于溢出问题，无法对 x, y 直接求和，所以构造 $\text{int avg} = (x \& y) + ((x \wedge y) \gg 1)$;这样可以避免 x, y 的直接加和，经检验对负数也成立。但是规则要求正中间时靠近第一个参数 x ，所以要加入对应判断条件

```

int sx = x >> 31;
int sy = y >> 31;
int same = ~(sx ^ sy);
int diff = x + ~y + 1;
int gt = (~sx & sy) | (same & ~(diff >> 31));
int odd = (x ^ y) & 1;

```

这样就构建了当正中间的时候产生的数，其中 odd 用于判断和是否为奇数，如果为奇数，根据 x, y 的比较来判断到底加 1 还是不加。此时只要 $\text{return avg} + (\text{odd} \& \text{gt})$;将 odd 和构建的 gt 进行&就可以判断。

P12.要求判断 x 是否在以 a, b 为端点的闭区间内，端点顺序任意。逻辑上就是: $(a \leq x \&\& x \leq b) \mid \mid (b \leq x \&\& x \leq a)$ ，但不能直接用 $\leq$ ，也不能直接做减法，因为 $x - a$ 可能溢出。所以经过尝试后构建 $\text{int less} = ((x - y) \wedge ((x \wedge y) \& ((x - y) \wedge x))) \gg 31$;此时如果 $x < y$ ，less 为全 1；否则为 0。将减法用加法转换，然后用 ! 把小于变成大于等于，再组合两个区间。最后即为

```

int d1 = x + ~a + 1;
int l1 = (d1 ^ ((x ^ a) & (d1 ^ x))) >> 31;
int d2 = b + ~x + 1;
int l2 = (d2 ^ ((b ^ x) & (d2 ^ b))) >> 31;
int d3 = x + ~b + 1;
int l3 = (d3 ^ ((x ^ b) & (d3 ^ x))) >> 31;
int d4 = a + ~x + 1;
int l4 = (d4 ^ ((a ^ x) & (d4 ^ a))) >> 31;

```

```
int c1 = (!l1) & (!l2); // a <= x <= b
int c2 = (!l3) & (!l4); // b <= x <= a
return c1 | c2;
```

P13.直接算即为 $\text{int } p = (x \ll 2) + x$

然后判断溢出：正溢出： x 非负且 $x > \text{INT_MAX} / 5$ ；负溢出： x 为负且 $x < \text{INT_MIN} / 5$ 。
 阈值 $t = \text{INT_MAX} / 5 = 0x19999999$ 。通过构建掩码来进行判断和输出数字。正溢出掩码时不能直接用 $x + \sim t$ 的符号位，因为 x 是负数时可能溢出导致误判。所以要先屏蔽负数：
 $\text{int pos} = \sim((x \gg 31) | ((x + \sim t) \gg 31));$ 然后负溢出掩码， $\text{int neg} = ((x + t) \gg 31) \& (x \gg 31);$ 然后构建 mask 掩码，最后 return 在结果，正溢出和负溢出中选择。

```
int mask = pos | neg;
int imin = 1 << 31;
int imax = ~imin;
return (p & ~mask) | (pos & imax) | (neg & imin);
```

P14.要求判断 $x + y + z$ 是大于 INT_MAX 、小于 INT_MIN ，还是落在范围内。

要把加法拆成两步： $t = x + y$ ，检测第一次溢出方向； $u = t + z$ ，检测第二次溢出方向。

每次溢出的方向可以用符号位判断：两个正数相加得到负数 $\rightarrow$ 正溢出；

两个负数相加得到正数 $\rightarrow$ 负溢出。然后把两次溢出的“方向”相加：如果最终方向为正，返回 1；如果最终方向为负，返回 -1；否则返回 0。其中

P15.首先提取符号、阶码、尾数

```
unsigned sign = uf & 0x80000000;
unsigned exp = (uf >> 23) & 0xFF;
unsigned frac = uf & 0x7FFFFFFF;
```

然后进行特殊值处理

```
if (exp == 0xFF) return uf;
if (exp == 0) {
```

```
    if (frac == 0) return uf;
```

然后进行非规格化数处理

```
    unsigned T = frac * 3;
    unsigned N = T >> 1;
    if ((T & 1) && (N & 1)) N++;
```

if $((T \& 1) \&\& (N \& 1)) N++;$ ：最近偶数舍入。 $T \& 1$ 表示除以 2 有余数， $N \& 1$ 表示当前整数部分为奇数。如果正好是 0.5 且整数部分为奇数，就进位到偶数。

```
    if (N >= (1 << 23)) {
        exp = 1;
        frac = N - (1 << 23);
    } else {
        exp = 0;
        frac = N;
    }
```

```
    return sign | (exp << 23) | frac;
```

如果 N 达到了 2^{23} ，说明结果不再是非规格化数，而是变成了最小的规格化数：阶码为 1，尾数为 $N - 2^{23}$

最后进行规格化处理：

```
    if (T >= (1 << 25)) {
```

```

    E1 = exp + 1;
    N = T >> 2;
    unsigned r = T & 3;
    if (r > 2 || (r == 2 && (N & 1))) N++;
} else {
    N = T >> 1;
    unsigned r = T & 1;
    if (r && (N & 1)) N++;
}

```

乘 3 后，有效数字可能进位到 25 位。把它回到 24 位。

乘 3 后没有超过 2^{25} ，只需要除以 2 一次。

```

    if (N >= (1 << 24)) {
        N >>= 1;
        E1++;
    }

```

舍入后 N 可能达到 25 位，这时需要再右移一位，阶码再加 1，保持规格化。

```

    if (E1 >= 255) return sign | 0x7F800000;
    return sign | (E1 << 23) | (N & 0x7FFFFFFF);

```

P16.16 与 15 一些方面类似，但核心步骤不同

16 题先算实际指数 $e = \text{exp} - 127$ 。如果 $e \geq 23$ ，说明小数部分已经不存在，数值本身就是整数，直接返回原值。否则需要右移 $\text{shift} = 23 - e$ 位，把尾数中的小数部分移出去。

右移后得到整数部分 N，被移出的低位是余数，用最近偶数舍入决定 N 是否加 1。如果舍入后 $N == 0$ ，返回带符号的 0。否则，把整数 N 重新转回浮点表示：找 N 的最高位，确定新阶码，再构造尾数，最后加上原来的符号位。

P17.17 题先处理 0: $x == 0$ 直接返回 0。如果是负数，记录符号位，并取绝对值。找绝对值最高位的位置 k，阶码就是 $127 + k$ 。如果 $k \leq 23$ ，有效数字不超过 24 位，尾数直接左移对齐到 23 位。如果 $k > 23$ ，右移 $k - 23$ 位，保留高 24 位有效数字，被移出的低位用最近偶数舍入。舍入后如果有效数字进位到 25 位，就再右移一位，阶码加 1。最后拼上符号、阶码和尾数。

P18. 可以用循环方法简单得到答案，但是在不用循环方法时由于一个一个处理 1 的个数会导致操作符达到上限，所以需要构建掩码来分布处理。

构建掩码

```

int m1 = (0x55 << 8) | 0x55;
m1 = (m1 << 16) | m1;
int m2 = (0x33 << 8) | 0x33;
m2 = (m2 << 16) | m2;
int m4 = (0x0F << 8) | 0x0F;
m4 = (m4 << 16) | m4;
int m8 = (0xFF << 16) | 0xFF;
int m16 = (0xFF << 8) | 0xFF;

```

然后进行

```

x = (x & m1) + ((x >> 1) & m1);
x = (x & m2) + ((x >> 2) & m2);
x = (x & m4) + ((x >> 4) & m4);

```

```
x = (x & m8) + ((x >> 8) & m8);  
x = (x & m16) + ((x >> 16) & m16);
```

由于这样的操作会

每 2 位一组，统计 1 的个数；

每 4 位合并两个 2 位计数；

每 8 位合并两个 4 位计数；

每 16 位合并两个 8 位计数；

最后合并高低 16 位。

本身掩码的构建较难联想到，但是如果从整体思路出发可以想出这样的每一步操作从而想出掩码，因为不采用循环的方法只能采用整体的思路构建。

P19

要求反转 32 位。

思路是

交换相邻 1 位；

交换相邻 2 位块；

交换相邻 4 位块；

交换相邻 8 位块；

交换高低 16 位。

每一步都用到掩码：

```
x = ((x >> 1) & m1) | ((x & m1) << 1);
```

```
x = ((x >> 2) & m2) | ((x & m2) << 2);
```

```
x = ((x >> 4) & m4) | ((x & m4) << 4);
```

```
x = ((x >> 8) & m8) | ((x & m8) << 8);
```

```
x = ((x >> 16) & m16) | (x << 16);
```

这和 P18 类似，本身掩码的构建较难联想到，但是如果从整体思路出发可以想出这样的每一步操作从而想出掩码。

